CÓDIGOCLARO · RUTA COMPLEMENTARIA

TypeScript: de JavaScript a React y Node

15 temas · Para quienes ya conocen JavaScript · Revisión documental: 27 de octubre de 2026

Propósito de la ruta

El recorrido parte de funciones y objetos JavaScript y añade, paso a paso, el sistema de tipos de TypeScript. Termina usando los contratos en módulos orientados a Node y en componentes React con TSX. La documentación oficial enlazada guía cada tema; los ejemplos y prácticas de este material están redactados para este curso.

Cómo trabajar

En cada tema, predice primero qué acepta el verificador. Después resuelve la práctica y compara el resultado con el criterio de dominio. Las comprobaciones estáticas describen el programa; valida en ejecución las entradas externas y consulta la configuración del proyecto antes de aplicar opciones a Node o React.

Tema 01 · Del JavaScript al verificador de TypeScript

TypeScript analiza estáticamente programas JavaScript y permite describir sus valores con tipos. El análisis ayuda a detectar incompatibilidades antes de ejecutar; no inspecciona ni valida por sí solo los datos que llegan en tiempo de ejecución.

Objetivo: Distinguir verificación estática, ejecución JavaScript y opciones del proyecto; reconocer qué cambia al activar strict.

Práctica: Toma una función JavaScript que calcule el total de una compra. Añade tipos gradualmente, configura un tsconfig mínimo con strict y registra qué defectos detecta el compilador y cuáles requieren comprobaciones en ejecución.

Criterio de dominio: Explica un hallazgo estático y un riesgo que sigue en tiempo de ejecución.
Documentación: Handbook: introducción · Referencia oficial de TSConfig

Tema 02 · Inferencia, anotaciones y tipos básicos

El compilador puede inferir un tipo a partir del valor inicial y del contexto. Las anotaciones son útiles cuando aclaran una frontera o un contrato; no hace falta repetir automáticamente información que ya está bien inferida.

Objetivo: Leer tipos inferidos y usar string, number, boolean y tipos literales donde aclaren el contrato.

Práctica: Convierte una función de descuento de JS: acepta un precio y un código promocional limitado a "ESTUDIANTE" o "GENERAL". Anota solo las fronteras necesarias y compara los errores con la versión sin tipos.

Criterio de dominio: Predice qué llamadas son aceptadas sin recurrir a any.
Documentación: Handbook: tipos cotidianos

Tema 03 · Arrays, tuplas y colecciones de solo lectura

Los arrays representan secuencias de elementos de un tipo; las tuplas expresan una secuencia con posiciones y tipos conocidos. Las variantes readonly limitan las operaciones de escritura disponibles desde esa referencia.

Objetivo: Elegir entre array y tupla según si importa el tamaño o el significado de cada posición.

Práctica: Modela filas de un CSV de inventario como pares [codigo, unidades]. Escribe una función que sume unidades por código y explica por qué su entrada es una colección y cada fila una tupla.

Criterio de dominio: Detecta una fila incompleta y evita mutar la entrada si el contrato la declara readonly.
Documentación: Handbook: tipos cotidianos · Handbook: tipos de objeto

Tema 04 · Tipos de objeto y propiedades opcionales

Un tipo de objeto describe las propiedades que una operación puede leer. `?` indica que una propiedad puede faltar; `readonly` restringe la asignación a través de esa referencia, pero no convierte mágicamente todo el objeto en inmutable.

Objetivo: Describir registros con propiedades requeridas, opcionales y de solo lectura.

Práctica: Define `Modulo` con identificador readonly, nombre, curso opcional y plazas. Implementa una función que presente un curso ausente como "sin asignar" y revisa un objeto literal con una propiedad mal escrita.

Criterio de dominio: No confunde una propiedad opcional con una propiedad que siempre contiene undefined.
Documentación: Handbook: tipos de objeto

Tema 05 · Uniones y narrowing del flujo de control

Una unión expresa varios estados posibles. Comprobaciones como typeof, in y una propiedad discriminante permiten al compilador estrechar esa unión dentro de una rama; la lógica de ejecución sigue siendo necesaria.

Objetivo: Representar alternativas sin campos contradictorios y estrecharlas antes de usar propiedades específicas.

Práctica: Modela el resultado de buscar un módulo con los estados encontrado, no encontrado y error. Usa `kind` y, en `default`, asigna el valor restante a una variable de tipo `never`; añade un estado y comprueba que el compilador señala esa asignación hasta que incluyas su rama.

Criterio de dominio: Cada rama accede solo a los datos presentes en ese estado.

Documentación: Handbook: narrowing · Handbook: tipos cotidianos

Tema 06 · Funciones, callbacks y contratos

Los tipos de función describen parámetros y resultado. Los callbacks pueden recibir sus propios contratos; parámetros opcionales y rest deben reflejar las llamadas que el código realmente permite.

Objetivo: Tipar entradas, salidas y callbacks sin usar Function ni debilitar el contrato con any.

Práctica: Escribe `filtrarModulos` con una lista y un callback de predicado. Añade un callback que compruebe curso y otro que acepte un criterio opcional; documenta qué devuelve la función con una lista vacía.

Criterio de dominio: El tipo del callback permite leer el módulo y devuelve un booleano.

Documentación: Handbook: funciones

Tema 07 · type, interface y composición de contratos

Los alias dan nombre a tipos y pueden representar uniones, primitivas u objetos. Las interfaces describen formas de objeto y pueden extender otras interfaces. La compatibilidad de objetos de TypeScript se basa normalmente en la forma requerida.

Objetivo: Escoger una forma legible de compartir y ampliar contratos de datos.

Práctica: Define `Persona` y `Docente`; expresa que un docente conserva nombre e identificador y añade departamento. Crea una función que reciba `Persona` y analiza por qué también acepta un docente.

Criterio de dominio: Justifica la composición por los campos requeridos, no por el nombre del tipo.

Documentación: Handbook: tipos de objeto

Tema 08 · null, unknown y límites de confianza

Con strictNullChecks, null y undefined deben tratarse explícitamente. unknown admite un valor no conocido, pero exige estrecharlo antes de operar. Un tipo estático no demuestra que una respuesta JSON cumpla el esquema esperado.

Objetivo: Modelar ausencia y estrechar valores externos antes de tratarlos como datos de dominio.

Práctica: Escribe `leerModulo(valor: unknown)` que acepte solo un objeto con `id` string y `nombre` string. Comprueba null, una cadena, un objeto incompleto y uno válido; devuelve un resultado explícito para cada caso.

Criterio de dominio: La función valida en ejecución y no convierte datos con una aserción ciega.

Documentación: Handbook: tipos cotidianos · Handbook: narrowing · Referencia oficial de TSConfig

Tema 09 · Genéricos y restricciones

Un parámetro genérico conserva la relación entre tipos de entrada y salida. Una restricción expresa qué propiedades necesita una operación sin eliminar el tipo concreto del valor recibido.

Objetivo: Crear funciones reutilizables que preserven el tipo de su entrada y limiten solo lo necesario.

Práctica: Implementa `buscarPorId<T extends { id: string }>` para una colección de módulos o profesores. Devuelve el elemento con su tipo original y representa explícitamente el caso sin coincidencia.

Criterio de dominio: El resultado mantiene campos propios de T sin añadir aserciones.
Documentación: Handbook: genéricos

Tema 10 · Tipos derivados y operadores de tipos

`keyof` obtiene claves de un tipo y el acceso indexado consulta el tipo asociado a una clave. Los tipos de utilidad Pick y Partial expresan transformaciones comunes sobre esos contratos.

Objetivo: Derivar una forma de actualización sin duplicar el tipo completo del dominio.

Práctica: Construye `CambiosDeModulo` a partir de `Modulo`: solo se pueden cambiar nombre y plazas, ambos opcionales. Usa el tipo en una función de actualización y provoca un error intentando editar el id.

Criterio de dominio: El tipo derivado acepta cambios permitidos y rechaza `id` en un objeto literal comprobado.
Documentación: Handbook: creación de tipos a partir de tipos · Handbook: tipos de utilidad

Tema 11 · Clases y compatibilidad estructural

Las clases añaden constructores, campos y métodos; no son la única forma de describir objetos. El sistema de tipos estructural permite que valores distintos sean compatibles cuando ofrecen los miembros requeridos.

Objetivo: Separar un contrato de servicio de una implementación concreta y evitar jerarquías sin una relación de sustitución clara.

Práctica: Declara un contrato `RepositorioModulo` y crea una clase que lo implemente. Pasa también un objeto literal con los mismos métodos; compara qué garantiza el contrato y qué comportamiento debe comprobarse aparte.

Criterio de dominio: El consumidor depende del contrato y no de campos internos de la clase.
Documentación: Handbook: clases · Handbook: tipos de objeto

Tema 12 · Módulos TypeScript en un proyecto Node

import y export describen dependencias entre módulos. El compilador configura cómo interpreta y emite módulos; el cargador del entorno ejecuta y resuelve dependencias. El formato elegido debe concordar con el runtime objetivo.

Objetivo: Entender por qué module, moduleResolution y las convenciones del entorno Node afectan a la resolución de importaciones.

Práctica: Separa un modelo de módulo, una función de búsqueda y el punto de entrada en archivos distintos. Sigue cada importación y consulta TSConfig para anotar qué decisión debe coincidir con el Node objetivo antes de ejecutar.

Criterio de dominio: Distingue una comprobación del compilador de la resolución real del cargador.
Documentación: Handbook: módulos · Referencia oficial de TSConfig

Tema 13 · Migración gradual de JavaScript y declaraciones

La migración puede admitir archivos JS junto a TS mediante allowJs; JSDoc aporta información de tipos a archivos JavaScript. Los archivos .d.ts describen una API existente y no implementan su comportamiento.

Objetivo: Planificar una conversión por módulos y decidir cuándo basta JSDoc o hace falta una declaración.

Práctica: Elige un helper JS sin dependencias externas. Activa `allowJs` e incluye el archivo en tsconfig; añade JSDoc para sus parámetros y retorno y conviértelo luego a TS conservando las llamadas. Redacta una declaración mínima para una dependencia sin tipos.

Criterio de dominio: La migración mantiene la API y separa descripción de implementación.

Documentación: Handbook: migrar desde JavaScript · Handbook: tipos JSDoc · Handbook: archivos de declaración

Tema 14 · React con TSX: props, children y eventos

Los archivos que contienen JSX usan .tsx y necesitan opciones JSX válidas. TypeScript puede inferir el tipo del evento por el contexto del handler JSX; los tipos de React ofrecen ChangeEvent y ReactNode para children.

Objetivo: Definir props explícitas y tratar el evento adecuado sin anotar cada expresión por costumbre.

Práctica: Crea `FiltroModulo.tsx` con props para texto, valor y callback. Conecta un input a `React.ChangeEvent<HTMLInputElement>`, ofrece children opcionales como `React.ReactNode` y consulta la guía para `@types/react`, `@types/react-dom` y una opción `jsx` válida en tsconfig.

Criterio de dominio: Una llamada JSX inválida se detecta y el callback expone el valor correcto.

Documentación: React: usar TypeScript

Tema 15 · React con estado tipado y estados de petición

useState suele inferir su tipo inicial. Una unión discriminada puede representar estados mutuamente excluyentes de una petición; useReducer acepta un estado y acciones que pueden describirse con tipos.

Objetivo: Evitar combinaciones de estado imposibles y estrechar el estado antes de mostrar datos o errores.

Práctica: Construye una pantalla de módulos con idle, loading, success y error. Tipifica estado y acciones, implementa un reducer con switch y renderiza los datos solo en success; añade una acción para volver a intentar.

Criterio de dominio: Cada transición conserva los campos que corresponden a su estado.

Documentación: React: usar TypeScript · Handbook: narrowing

Laboratorio integrador · catálogo de módulos

Organiza una solución en tres piezas: un módulo compartido con el tipo `Modulo` y funciones puras; una entrada Node que importe esas funciones; y un componente React que reciba resultados mediante props y represente estados de petición con una unión discriminada. Antes de aceptar JSON como `Modulo`, valida sus campos en tiempo de ejecución. Entrega los tres módulos, un caso normal, uno vacío y uno inválido, y explica qué garantiza el verificador y qué comprobación protege la frontera de datos.

Documentación: Handbook: módulos · React: usar TypeScript · Handbook: narrowing